# LLM fundamentals

Understand how large language models work in practice: tokens, context windows, sampling, training, API calls, cost, latency, evaluation and debugging.

A MARGA printable pack. Free, offline, no sign-up. Learn online at marga.eknathalabs.com.

MARGA by EknathaLabs
marga.eknathalabs.com

## Contents

1. What an LLM is and how it predicts tokens (11 min)
2. Tokens, context windows and limits (12 min)
3. Sampling, temperature and determinism (11 min)
4. Training, knowledge and hallucination (12 min)
5. Calling an LLM API (13 min)
6. Cost, latency and caching (12 min)
7. Choosing and evaluating models (12 min)
8. Debugging LLM applications (14 min)
9. Embeddings and vector search (12 min)
10. Retrieval-augmented generation (RAG) (13 min)
11. Prompting, RAG or fine-tuning: choosing (12 min)
12. LLM safety, privacy and data handling (13 min)
Cheat sheet

# Lesson 1. What an LLM is and how it predicts tokens

11 minutes. See a large language model as a next-token predictor, and understand what that means for what it does well and badly.

A large language model, or LLM, is a program trained on a very large amount of text to predict what comes next. That one skill, repeated thousands of times, produces answers, code, summaries and conversation. Engineers who understand the prediction loop make better decisions about prompts, costs and safety than those who treat the model as a mind or as a search engine.

## What you will learn

- The prediction loop: text in, one token out, repeat
- Why the same prompt can give different answers
- What models are good at, and where they fail
- A mental model you can use when something goes wrong

## The loop

The model receives text, converted to numbers, and produces a score for every possible next piece of text. One piece is chosen, added to the input, and the process repeats until the model produces a stop signal or hits a length limit. Nothing else happens inside: no lookup in a database and no running of code unless you give it tools. A reply of 300 words is hundreds of small predictions in a row, each depending on everything before it.

## Why it feels smart

Predicting text well requires capturing grammar, facts, style, reasoning patterns and code structure that appear in the training text. So the model can explain a stack trace, translate a paragraph or write a Terraform module, because those patterns were common in what it learned. It does not "look up" an answer. It produces the most plausible continuation, shaped by training to be helpful and honest.

## Strengths and weak spots

- **Strong**: summarising, rewriting, classifying, extracting fields from messy text, drafting code and explanations, translating between formats.
- **Weaker**: exact arithmetic on long numbers, counting characters, facts after its training cutoff, rarely seen details (an obscure flag of a tool), and anything that needs real-time data.
- **Risky**: confident but wrong answers, called hallucinations. A fluent sentence is not evidence of a true one.

The practical rule: use the model for language and pattern work, and use tools (a calculator, a search, a database) for exact facts.

## A mental model that helps debugging

When a result is wrong, ask three questions. **What did the model actually see?** Print the exact input, including the system prompt and any retrieved text. **Is the answer in the input?** If not, the model must guess. **Is the task a language task?** If it needs exactness, add a tool or a check. Most "model problems" are input problems. Keep the model's output as untrusted data: validate it before it reaches a database, a shell or a customer.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: How What an LLM is and how it predicts tokens fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **The loop** | The model receives text, converted to numbers, and produces a score for every possible next piece of text. | Asking the model for exact arithmetic or recent facts without tools. |
| **Why it feels smart** | Predicting text well requires capturing grammar, facts, style, reasoning patterns and code structure that appear in the training text. | Not looking at the exact input the model received. |
| **Strengths and weak spots** | Strong: summarising, rewriting, classifying, extracting fields from messy text, drafting code and explanations, translating between formats. | Putting model output straight into a shell or database. |
| **A mental model that helps debugging** | When a result is wrong, ask three questions. | Treating fluent output as verified output. |

Flow: The loop to Why it feels smart / Why it feels smart to Strengths and weak spots / Strengths and weak spots to A mental model that helps debugging

## Incident: The confident wrong flag

**Incident:** *An on-call engineer pasted a model answer into a runbook without checking it.*

```
$ kubectl rollout restart deploy/web --graceful-period=30
error: unknown flag: --graceful-period
$ kubectl rollout restart --help | grep -i grace
(no output)
```

The model had produced a flag that looks right but does not exist for that command. The runbook was corrected using the tool's own help, and the team added a rule: commands from a model are tested in a safe environment and checked against the official reference before they enter documentation.

## Common mistakes

- Treating fluent output as verified output
- Asking the model for exact arithmetic or recent facts without tools
- Not looking at the exact input the model received
- Putting model output straight into a shell or database

### Check yourself

**What does an LLM do at each step?**

Answer: Predicts the next token from the text so far

**Why can the same prompt give different answers?**

Answer: The next token is sampled from probabilities

**Which task is best done by code, not the model alone?**

Answer: Computing a checksum

**What is a hallucination?**

Answer: A fluent answer that is not true or not supported by the input

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 2. Tokens, context windows and limits

12 minutes. Count tokens, understand what fills a context window, and design around input and output limits.

Models do not read characters or words. They read tokens, small pieces of text, and every limit and every price is counted in tokens. Knowing what a token is and what fills the context window explains why long chats get worse, why prices depend on length, and why a prompt that works on a short example can fail on real data.

## What you will learn

- What a token is, and rough sizes
- What counts toward the context window
- Input limits versus output limits
- Strategies for text that does not fit

## Tokens

A tokenizer splits text into pieces such as whole common words, parts of rare words, punctuation and spaces. As a rule of thumb, one token is about four characters of English text, or three quarters of a word. Code, other languages and unusual strings often need more tokens per character. Different model families use different tokenizers, so counts differ. Use the provider's token counting tool rather than guessing when limits matter.

## The context window

The context window is the total number of tokens the model can consider in one request: the system prompt, the conversation so far, any documents or tool results you include, and the answer being generated. When the sum is too large, the request fails or older text must be dropped. Long contexts also cost more and can reduce accuracy for details buried in the middle, so more text is not always better.

## Input and output limits

Providers set a maximum number of output tokens per reply, separate from the context size. If you ask for a long report with a small output limit, the answer is cut off, and the response says it stopped because of length. Always check the stop reason in the response. A truncated JSON document is invalid JSON, which breaks the program that parses it.

```
response = client.messages.create(
    model=MODEL,
    max_tokens=1024,
    messages=[{"role": "user", "content": prompt}],
)
if response.stop_reason == "max_tokens":
    raise RuntimeError("answer was cut off; raise max_tokens or shorten the task")
```

## When text does not fit

- **Select**: send only relevant parts, found by search or by simple rules.
- **Summarise**: compress old conversation turns into a short summary.
- **Chunk**: process a long document in pieces and combine the results.
- **Cache**: reuse a large, unchanging prefix, such as instructions or a reference document, using prompt caching to cut cost and delay.
- **Measure**: log input and output token counts for each request, so cost and growth are visible.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Tokens, context windows and limits fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Tokens** | A tokenizer splits text into pieces such as whole common words, parts of rare words, punctuation and spaces. | Guessing token counts instead of measuring. |
| **The context window** | The context window is the total number of tokens the model can consider in one request: the system prompt, the conversation so far, any documents or tool results you... | Putting entire documents in when a few paragraphs would do. |
| **Input and output limits** | Providers set a maximum number of output tokens per reply, separate from the context size. | Ignoring the stop reason. |
| **When text does not fit** | Select: send only relevant parts, found by search or by simple rules. | Resending unlimited history. |

Flow: Tokens to The context window / The context window to Input and output limits / Input and output limits to When text does not fit

## Incident: The report that stopped in the middle

**Incident:** *A nightly job that asked a model for a JSON report failed to parse its output.*

```
$ python summarize.py
json.decoder.JSONDecodeError: Unterminated string starting at: line 41 column 12
$ python -c "print(resp.stop_reason, resp.usage.output_tokens)"
max_tokens 1024
```

The model hit the output limit in the middle of the JSON. The fix raised max_tokens, asked for shorter field values, and added a check that raises an error when the stop reason is max_tokens instead of passing broken text to the parser.

## Common mistakes

- Guessing token counts instead of measuring
- Ignoring the stop reason
- Resending unlimited history
- Putting entire documents in when a few paragraphs would do

### Check yourself

**Roughly how many characters is a token in English?**

Answer: About four

**What counts toward the context window?**

Answer: System prompt, history, documents, tool results and the answer

**What should you check when JSON output fails to parse?**

Answer: The stop reason, in case the output was cut off

**How can you cut the cost of a large unchanging prefix?**

Answer: Use prompt caching

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 3. Sampling, temperature and determinism

11 minutes. Understand how the next token is chosen, what temperature and top-p change, and how to get repeatable behaviour from a system that is random by design.

After the model scores every possible next token, something must choose one. That choice is called sampling, and it explains why answers vary between runs. Getting predictable behaviour for a pipeline is partly a matter of settings, and mostly a matter of designing the system so that small variation does not matter.

## What you will learn

- How sampling chooses a token
- What temperature and top-p do
- What you can and cannot make deterministic
- Designing around variation

## Probabilities

For each step the model produces a probability for every token. If "Paris" has 92 percent and "Lyon" has 3 percent after "The capital of France is", then most runs choose Paris. Different prompts and different points in a sentence have very different spreads: a factual answer is peaked, a story opening is wide.

## Temperature and top-p

**Temperature** rescales probabilities before choosing. Low values (near 0) make the highest-probability token win almost always, giving focused and repeatable output. Higher values flatten the choice and give more varied, creative output. **Top-p** limits the choice to the smallest set of tokens whose probabilities add up to p. Change one of them, not both, unless you have a reason. For extraction, classification and code, use low temperature; for brainstorming, use higher.

```
response = client.messages.create(
    model=MODEL,
    max_tokens=300,
    temperature=0.0,
    messages=[{"role": "user", "content": "Classify this ticket: " + ticket}],
)
```

## Determinism has limits

Even at temperature 0, results can differ slightly between calls, because of how requests are batched and how hardware does arithmetic, and any change to the model version changes outputs. Some newer models and modes fix or ignore sampling settings, so check the documentation for the model you use. Never write tests that require an exact sentence from a model, unless you have pinned the model and accept occasional breaks.

## Designing around variation

- Ask for **structured output** and validate it; the format stays stable even when wording varies.
- Test **properties**, not strings: the answer must contain a valid category, a number in range, or cite a given document.
- Run an important test several times and look at the **pass rate**.
- For decisions, use a constrained set of answers and a fallback to a human when the model is unsure.
- Pin a **model version** in production and re-run your tests before upgrading.

MARGA by EknathaLabs
marga.eknathalabs.com

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Sampling, temperature and determinism fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Probabilities** | For each step the model produces a probability for every token. | Not pinning the model version. |
| **Temperature and top-p** | Temperature rescales probabilities before choosing. | Using high temperature for extraction or classification. |
| **Determinism has limits** | Even at temperature 0, results can differ slightly between calls, because of how requests are batched and how hardware does arithmetic, and any change to the model... | Expecting exact repeatability. |
| **Designing around variation** | Ask for structured output and validate it; the format stays stable even when wording varies. | Check the Common mistakes list below this diagram for what usually goes wrong here. |

Flow: Probabilities to Temperature and top-p / Temperature and top-p to Determinism has limits / Determinism has limits to Designing around variation

## Incident: The flaky classifier

**Incident:** *A ticket router sent the same message to different queues on different days.*

```
$ for i in 1 2 3 4 5; do python route.py "refund not received"; done
billing
billing
payments
billing
payments
```

The prompt asked for free text, the temperature was left at the default, and "payments" and "billing" both seemed reasonable. The team listed the allowed queues in the prompt, requested one value from that list in JSON, set a low temperature, and added a test that runs each sample ten times.

## Common mistakes

- Using high temperature for extraction or classification
- Expecting exact repeatability
- Tuning temperature and top-p together without measuring
- Not pinning the model version

### Check yourself

**What does a low temperature do?**

Answer: Makes the most likely token win almost always

**What is top-p?**

Answer: A limit to the smallest set of tokens whose probabilities add up to p

**Is temperature 0 a guarantee of identical output?**

Answer: No, small differences can still occur

**How should a test of a model-based feature be written?**

Answer: Check properties of the answer, repeated several times

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 4. Training, knowledge and hallucination

12 minutes. Learn how a model gets its knowledge, why it has a cutoff date, and the practical ways to supply facts it does not have.

A model's knowledge comes from training, and training ended at some date. Your documents, your tickets and yesterday's news were never part of it. Understanding the stages of training, and the difference between knowledge in the model and knowledge in the prompt, tells you when to use context, retrieval, tools or training.

## What you will learn

- Pre-training and post-training in plain terms
- The knowledge cutoff and why facts go stale
- Why hallucinations happen
- Ways to add knowledge: context, retrieval, tools, fine-tuning

## Stages of training

**Pre-training** teaches the model to predict text from a huge collection of documents and code. This gives it language, broad facts and skills. **Post-training** then shapes behaviour with examples and feedback so that the model follows instructions, refuses harmful requests and writes in a helpful style. Together they produce an assistant. The details differ by provider, and are not something you control when you call an API.

## The cutoff

The training data ends at a date, called the knowledge cutoff. After that, the model knows nothing unless you provide it. This affects library versions, product names, prices, laws and current events. A model may also be unsure about its own cutoff, so do not ask it what is current. Put the current date and the facts you need into the prompt, or give it a search tool.

## Why it makes things up

The model is trained to produce plausible text, and when the answer is not in its training or in your prompt, a plausible-sounding answer is still the most likely continuation. Rare facts, precise numbers, citations and names of obscure APIs are the most common places it invents. You reduce this by giving source text and asking for answers only from it, allowing "I don't know" as a valid answer, and asking for quotes that you can check.

## Adding knowledge

1. **Put it in the prompt** (context): best for small, current, private facts.
2. **Retrieve it**: search your documents and include the best passages (RAG).
3. **Give tools**: let the model call a database, an API or a search and use the result.
4. **Fine-tune**: train further on your examples. Useful for style, format or narrow tasks at scale; poor at adding frequently changing facts, and it costs effort to evaluate and maintain.

Start at the top of this list. Most applications never need the last one.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Training, knowledge and hallucination fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Stages of training** | Pre-training teaches the model to predict text from a huge collection of documents and code. | Assuming the model knows recent or private facts. |
| **The cutoff** | The training data ends at a date, called the knowledge cutoff. | Fine-tuning to add facts that change. |
| **Why it makes things up** | The model is trained to produce plausible text, and when the answer is not in its training or in your prompt, a plausible-sounding answer is still the most likely... | Not allowing the model to say it does not know. |
| **Adding knowledge** | Put it in the prompt (context): best for small, current, private facts. | Trusting citations without opening them. |

Flow: Stages of training to The cutoff / The cutoff to Why it makes things up / Why it makes things up to Adding knowledge

## Incident: The library that did not exist

**Incident:** *A model suggested a function that failed on first use.*

```
$ python -c "import boto3; boto3.client('s3').upload_stream('a', 'b')"
AttributeError: 'S3' object has no attribute 'upload_stream'
```

The model blended real method names into one that sounded right. The developer checked the official reference and found the correct call. The team now pastes the relevant documentation page into the prompt for unfamiliar APIs and reviews generated code with tests.

## Common mistakes

- Assuming the model knows recent or private facts
- Fine-tuning to add facts that change
- Not allowing the model to say it does not know
- Trusting citations without opening them

### Check yourself

**What is a knowledge cutoff?**

Answer: The date after which the training data has nothing

**Where should you start when the model lacks a fact?**

Answer: Put it in the prompt or retrieve it

**Which request reduces hallucination?**

Answer: Answer only from the provided text and say if it is not there

**When is fine-tuning a good fit?**

Answer: Consistent style or format on a narrow task at scale

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 5. Calling an LLM API

13 minutes. Make a first API call, understand messages and roles, stream replies, handle errors and rate limits, and keep keys safe.

Using a model from code is an ordinary HTTP API call with unusual inputs. The details that matter are the same as for any service: authentication, timeouts, retries, rate limits, logging and cost. Getting them right turns a demo into something that runs on a schedule at three in the morning.

## What you will learn

- The shape of a request: model, messages, limits
- Roles: system, user, assistant
- Streaming and timeouts
- Errors, rate limits, retries and key handling

## A first call

```
import os
import anthropic

MODEL = os.environ["MODEL_ID"]   # set the model name in configuration, not in code
client = anthropic.Anthropic()    # reads ANTHROPIC_API_KEY from the environment

response = client.messages.create(
    model=MODEL,
    max_tokens=500,
    system="You are a concise assistant for site reliability engineers.",
    messages=[{"role": "user", "content": "Explain a 502 error in two sentences."}],
)
print(response.content[0].text)
print(response.usage.input_tokens, response.usage.output_tokens)
```

A request names a model, a limit on output tokens, an optional system prompt and a list of messages. The reply has content blocks, a stop reason and token usage. Other providers have similar shapes with different field names, so wrap the call in your own small function.

## Roles and conversation state

Messages alternate between the **user** and the **assistant**. The **system** prompt sets rules and context for the whole conversation. The API is stateless: it remembers nothing between calls, so your program sends the whole history each time and decides what to keep, trim or summarise. Store conversations in your own database, and treat them as personal data.

## Streaming and timeouts

Long answers take seconds. Streaming returns tokens as they are generated, so users see text immediately and your server can stop early. Always set a client timeout, and handle a stream that ends in the middle.

```
with client.messages.stream(model=MODEL, max_tokens=500, messages=messages) as stream:
    for text in stream.text_stream:
        print(text, end="", flush=True)
```

## Errors, limits and keys

- **Rate limits** return a 429 status: slow down, and retry with exponential backoff and jitter. Respect any retry-after header.
- **Server errors** (500, 529 overloaded) are usually temporary: retry a few times, then fail clearly.
- **Client errors** (400, 401, 403) will not fix themselves: do not retry; log the message.

MARGA by EknathaLabs marga.eknathalabs.com

- **Keys** belong in a secret manager or environment variable, never in code, logs, client-side JavaScript or screenshots. Use separate keys per service and per environment, set spend limits, and rotate on suspicion.
- **Log** request IDs, model, token counts, latency and outcome, but not secrets or personal data.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

### Diagram: How Calling an LLM API fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **A first call** | A request names a model, a limit on output tokens, an optional system prompt and a list of messages. | Retrying client errors. |
| **Roles and conversation state** | Messages alternate between the user and the assistant. | Logging full prompts that contain personal data. |
| **Streaming and timeouts** | Long answers take seconds. Streaming returns tokens as they are generated, so users see text immediately and your server can stop early. | No timeout on API calls. |
| **Errors, limits and keys** | Rate limits return a 429 status: slow down, and retry with exponential backoff and jitter. | Hard-coding keys. |

Flow: A first call to Roles and conversation state / Roles and conversation state to Streaming and timeouts / Streaming and timeouts to Errors, limits and keys

## Incident: The key in the screenshot

**Incident:** *A teammate shared a terminal screenshot in a public channel that included an API key.*

```
$ env | grep -i anthropic
ANTHROPIC_API_KEY=sk-ant-XXXXXXXXXXXXXXXX
$ curl -s -o /dev/null -w "%{http_code}\n" https://api.example.com/v1/models -H "x-api-key: $OLD_K \
    EY"
401
```

The key was revoked as soon as the message was noticed, which is why the check now returns 401. A new key went into the secret manager, usage logs were reviewed for unexpected requests, and the team stopped printing environment variables in shared screens and added secret scanning to chat and repositories.

## Common mistakes

- Hard-coding keys
- Retrying client errors
- No timeout on API calls
- Logging full prompts that contain personal data

### Check yourself

**What does the system prompt do?**

Answer: Sets rules and context for the whole conversation

**Who keeps the conversation history?**

Answer: Your application, which resends it each call

**How should you react to a 429?**

Answer: Back off with jitter and retry

MARGA by EknathaLabs marga.eknathalabs.com

**Where does an API key belong?**

Answer: A secret manager or environment variable

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 6. Cost, latency and caching

12 minutes. Estimate what a feature costs, understand the parts of latency, and reduce both with model choice, prompt design and caching.

Model calls are priced per token and take noticeable time. A feature that is cheap in a demo can be expensive at ten thousand users a day, and slow enough to frustrate people. A little arithmetic before launch, and a few well-known techniques, keep both under control.

## What you will learn

- Estimate cost from tokens and traffic
- Time to first token and tokens per second
- Cut cost: smaller models, shorter prompts, caching, batching
- Budgets and alerts

## Back-of-envelope cost

Cost per request = input tokens times the input price, plus output tokens times the output price. Output tokens usually cost several times more than input tokens. Example with made-up prices: 3,000 input tokens at 3 dollars per million is 0.009 dollars, 500 output tokens at 15 dollars per million is 0.0075 dollars, so about 1.65 cents per request. At 50,000 requests per day that is about 825 dollars a day. Use your provider's current price list, and measure your real token counts.

```
def cost(inp, out, price_in=3.0, price_out=15.0):
    """Prices are dollars per million tokens (example values)."""
    return inp / 1e6 * price_in + out / 1e6 * price_out

print(round(cost(3000, 500) * 50_000, 2))
```

## Latency

Total time is roughly: network and queueing, **time to first token** (processing your input), and then output length divided by **tokens per second**. A long prompt delays the first token; a long answer takes long to finish. Streaming improves perceived speed. Smaller models are faster and cheaper, so use the smallest model that passes your tests for each step.

## Reducing cost and delay

- **Right-size the model**: easy steps (classify, extract) on a small model, hard steps on a large one.
- **Shorten**: trim instructions, remove unused context, ask for concise output or a fixed format.
- **Prompt caching**: mark a large stable prefix (instructions, reference text, tool definitions) so repeated requests reuse it at lower cost and delay. Put stable content first and changing content last.
- **Batch** non-urgent work through a batch interface, often at a discount.
- **Cache answers** to identical questions in your own store, if the content allows.
- **Limit** output with max_tokens and stop conditions.

## Budgets and visibility

Track tokens and cost per feature, per customer and per model. Set spend limits at the provider and alerts in your monitoring. Add a circuit breaker for runaway loops, especially in agents that call a model repeatedly. One forgotten retry loop can spend a month's budget overnight.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Cost, latency and caching fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Back-of-envelope cost** | Cost per request = input tokens times the input price, plus output tokens times the output price. | No token or cost tracking. |
| **Latency** | Total time is roughly: network and queueing, time to first token (processing your input), and then output length divided by tokens per second. | Using the largest model for every step. |
| **Reducing cost and delay** | Right-size the model: easy steps (classify, extract) on a small model, hard steps on a large one. | Putting changing text before the stable prefix. |
| **Budgets and visibility** | Track tokens and cost per feature, per customer and per model. | No budget limit on automated loops. |

Flow: Back-of-envelope cost to Latency / Latency to Reducing cost and delay / Reducing cost and delay to Budgets and visibility

## Incident: The overnight bill

**Incident:** *A scheduled job re-sent the same 40,000-token document 2,000 times.*

```
$ python usage_report.py --day yesterday
feature=doc-summary requests=2000 input_tokens=80,412,000 output_tokens=610,000
$ python usage_report.py --day yesterday --cached
cache_read_tokens=0
```

Nothing was cached, and the document was the same in every request. The team moved the document into a cached prefix, used a batch process for the nightly run, and added a daily spend alert and a hard stop at a set budget.

## Common mistakes

- No token or cost tracking
- Using the largest model for every step
- Putting changing text before the stable prefix
- No budget limit on automated loops

### Check yourself

**Which usually costs more per token?**

Answer: Output tokens

**What does prompt caching reuse?**

Answer: A stable prefix of the prompt

**How do you improve perceived latency?**

Answer: Stream the reply

**What protects you from a runaway loop?**

Answer: Spend limits, alerts and a circuit breaker

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 7. Choosing and evaluating models

12 minutes. Build a small test set from your own task, compare models and prompts with it, and avoid trusting public benchmarks alone.

Public benchmarks measure general ability. Your feature needs one specific thing done well on your data. The only reliable way to choose a model or a prompt is to test it on examples that look like your real traffic, and to keep those tests running as things change.

## What you will learn

- Why you need your own evaluation set
- Build a set of examples with expected properties
- Score outputs: exact checks, rules and model-based grading
- Compare, decide and keep testing

## Why your own tests

Benchmarks can be outdated, narrow, or leaked into training data, and they use tasks unlike yours. A model that ranks first on a leaderboard may do worse on your support tickets or your log format. A small set of 30 to 100 real, varied examples with known good answers will tell you more about your feature than any public score.

## Build the set

Collect real inputs (with personal data removed), include normal cases, hard cases, and cases where the right answer is "I don't know" or "escalate". For each, write what a good answer must contain or must not contain. Keep a held-out part you do not tune on. Store the set in version control next to the prompt.

```
{"id": "t-014", "input": "Customer says card was charged twice", "expect": {"category": "billing", \
     "must_not_say": ["refund approved"]}}
```

## Score the answers

- **Exact checks** for structured fields: category in the allowed list, number in range, valid JSON.
- **Rules** for text: contains required facts, avoids forbidden claims, stays under a length.
- **Model-based grading**: another model compares the answer to a rubric. Fast and useful, but it makes mistakes and has biases, so spot-check it with humans and prefer clear rubrics with yes or no questions.
- **Human review** of a sample, especially for tone and safety.

```
def score(case, output):
    ok = output.get("category") in ALLOWED
    ok = ok and output.get("category") == case["expect"]["category"]
    ok = ok and not any(w in output.get("reply", "") for w in case["expect"].get("must_not_say", [ \
    ]))
    return ok
```

## Decide and keep testing

Run each candidate model and prompt on the same set, several times, and compare pass rate, cost and latency. Choose the cheapest option that meets your quality bar. Re-run the set when you change the prompt, the model version, the retrieval, or the tools. Add every production failure as a new test case. This is a regression test for behaviour that cannot be unit tested in the old way.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Choosing and evaluating models fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Why your own tests** | Benchmarks can be outdated, narrow, or leaked into training data, and they use tasks unlike yours. | Choosing a model by leaderboard alone. |
| **Build the set** | Collect real inputs (with personal data removed), include normal cases, hard cases, and cases where the right answer is "I don't know" or "escalate". | Using only model-based grading with no human check. |
| **Score the answers** | Exact checks for structured fields: category in the allowed list, number in range, valid JSON. | Check the Common mistakes list below this diagram for what usually goes wrong here. |
| **Decide and keep testing** | Run each candidate model and prompt on the same set, several times, and compare pass rate, cost and latency. | Tuning on the same examples you evaluate on. |

Flow: Why your own tests to Build the set / Build the set to Score the answers / Score the answers to Decide and keep testing

## Incident: The upgrade that broke the router

**Incident:** *A model upgrade changed how the ticket router phrased its categories.*

```
$ python run_evals.py --model new --set tickets
pass_rate=0.71 (was 0.94)   failures: category_not_in_allowed_list=38
$ python run_evals.py --model old --set tickets
pass_rate=0.94
```

The new version added explanations to the category field, so parsing failed. Because the team had an evaluation set in CI, the problem showed up before release. They tightened the prompt and the schema, re-ran the set until the pass rate was back, and then switched gradually.

## Common mistakes

- Choosing a model by leaderboard alone
- Tuning on the same examples you evaluate on
- Using only model-based grading with no human check
- Not re-running tests after changes

### Check yourself

**What is the best basis for choosing a model for your feature?**

Answer: A test set built from your own task

**What should a test case contain?**

Answer: An input and what a good answer must or must not contain

**When should you re-run the evaluation?**

Answer: After changing the prompt, model, retrieval or tools

**Why spot-check model-based grading?**

Answer: Graders make mistakes and have biases

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 8. Debugging LLM applications

14 minutes. Investigate a bad answer step by step: capture the exact input, separate retrieval, prompt, model and parsing problems, and fix with tests.

When an LLM feature misbehaves there is no stack trace. The system is probabilistic and the failure may be in retrieval, the prompt, the model, the limits or the code around it. A fixed order of checks finds the cause quickly and turns each incident into a permanent test.

## What you will learn

- Capture the exact request and response
- Check limits, stop reasons and errors
- Separate retrieval, prompt, model and parsing problems
- Reproduce, fix and add a regression test

## Capture everything

You cannot debug what you cannot see. For every request, log the request ID, model and version, the full assembled prompt (with personal data protected), parameters, retrieved passages, tool calls and results, the raw reply, the stop reason, token counts and latency. Store them for a limited time with access controls. A trace that connects one user report to one stored request is worth more than any guess.

## First checks

```
jq '{id, model, stop_reason, usage, error}' logs/llm/2026-10-28/req-8f31.json
jq -r '.prompt' logs/llm/2026-10-28/req-8f31.json | head -60
jq -r '.reply' logs/llm/2026-10-28/req-8f31.json
```

Look for: an error or a **429**/**529**; **stop_reason** other than a normal end (cut off by length); the **model version** changed; the prompt contains stale or missing text; a very large input that pushed out instructions; the reply that is valid text but fails your parser.

## Which layer failed?

- **Retrieval or context**: is the needed fact in the prompt? If not, fix search, chunking or the data. The model cannot use what it never saw.
- **Prompt**: is the instruction clear, are examples consistent, do two rules conflict? Try the same input with a simplified prompt.
- **Model**: with the right facts and a clear prompt, does it still fail? Try a stronger model or break the task into steps.
- **Parsing and code**: is the output valid but your code mishandles it, for example by assuming the first content block is text?
- **Environment**: did a tool fail silently, or return an error that the model had to guess around?

## Fix and lock it in

Reproduce with the stored request, change one thing, and re-run. Then add the case to the evaluation set with the expected behaviour, so it cannot return unnoticed. Compare the new pass rate with the old on the whole set, not only on the case you fixed. Record the cause in your runbook: most incidents repeat. Add monitors for the signals that would have caught this earlier, such as parse-failure rate, empty-answer rate, refusal rate, token growth and user thumbs-down rate.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: How Debugging LLM applications fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Capture everything** | You cannot debug what you cannot see. For every request, log the request ID, model and version, the full assembled prompt (with personal data protected), parameters... | Blaming the model before checking the prompt contents. |
| **First checks** | Look for: an error or a 429/529; stop_reason other than a normal end (cut off by length); the model version changed; the prompt contains stale or missing text; a very... | Not logging the assembled prompt. |
| **Which layer failed?** | Retrieval or context: is the needed fact in the prompt? | The model cannot use what it never saw. |
| **Fix and lock it in** | Reproduce with the stored request, change one thing, and re-run. | Fixing one case without a regression test. |

Flow: Capture everything to First checks / First checks to Which layer failed? / Which layer failed? to Fix and lock it in

## Incident: The bot that stopped finding the policy

**Incident:** *After a documentation update, a support assistant began refusing policy questions.*

```
$ jq '{retrieved: [.retrieved[].source], stop_reason}' req-1204.json
{ "retrieved": ["blog/holiday-hours.md", "faq/shipping.md"], "stop_reason": "end_turn" }
$ jq '.retrieved[].source' req-0711-last-week.json
"policies/returns-v3.md"
```

The reindexing job had skipped the policies folder after a path change, so the returns policy was never retrieved and the model correctly said it did not know. The index was rebuilt, a monitor was added for documents that are not found for known test questions, and those questions became part of the nightly evaluation.

## Common mistakes

- Blaming the model before checking the prompt contents
- Not logging the assembled prompt
- Fixing one case without a regression test
- Changing several things at once

### Check yourself

**What should you check first for a bad answer?**

Answer: The exact assembled input and the stop reason

**The needed fact is absent from the prompt. Whose problem is it?**

Answer: Retrieval or context assembly

**What do you do after fixing a failure?**

Answer: Add it to the evaluation set

**Which signal can warn you early?**

Answer: Parse-failure or refusal rate rising

MARGA by EknathaLabs marga.eknathalabs.com

# Lesson 9. Embeddings and vector search

12 minutes. Turn text into vectors, search by meaning with similarity, and understand indexes, chunking and the limits of vector search.

Keyword search fails when people use different words for the same idea. Embeddings represent meaning as numbers, which lets you search by similarity. It is the base of retrieval for language model applications.

## What you will learn

- What an embedding is and how similarity works
- Chunking documents and storing vectors with metadata
- Approximate nearest neighbor indexes
- Hybrid search and evaluating retrieval

## Vectors and similarity

```
import numpy as np

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

docs = {"reset password": [0.9, 0.1, 0.0], "change billing card": [0.1, 0.9, 0.1]}
query = [0.85, 0.2, 0.05]            # "I forgot my login"
ranked = sorted(docs, key=lambda k: cosine(query, docs[k]), reverse=True)
print(ranked)
```

An embedding model returns a vector of hundreds or thousands of numbers. Texts with similar meaning get vectors that point in similar directions, and **cosine similarity** measures that angle.

## Chunking and metadata

Split documents into pieces of a few hundred tokens along natural boundaries such as headings, with a small overlap. Store each chunk with metadata: source, section, date, access group and version. Metadata lets you filter, cite and enforce permissions.

## Indexes

Comparing a query with millions of vectors is slow, so databases use **approximate nearest neighbor** indexes such as HNSW or IVF. They trade a little recall for large speed gains. Tune the search depth and measure recall against an exact search on a sample.

## Hybrid and evaluation

Pure vector search can miss exact terms like error codes and product names. **Hybrid search** combines keyword and vector scores. Build a small test set of real queries with known right documents and measure whether the right chunk appears in the top results, before and after each change to chunking or the model.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Embeddings and vector search fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Vectors and similarity** | An embedding model returns a vector of hundreds or thousands of numbers. | Using only vector search for codes and names. |
| **Chunking and metadata** | Split documents into pieces of a few hundred tokens along natural boundaries such as headings, with a small overlap. | No metadata or access filters. |
| **Indexes** | Comparing a query with millions of vectors is slow, so databases use approximate nearest neighbor indexes such as HNSW or IVF. | Chunks too large or cut mid-sentence. |
| **Hybrid and evaluation** | Pure vector search can miss exact terms like error codes and product names. | Check the Common mistakes list below this diagram for what usually goes wrong here. |

Flow: Vectors and similarity to Chunking and metadata / Chunking and metadata to Indexes / Indexes to Hybrid and evaluation

## Incident: The right document ranked tenth

**Incident:** *A support assistant could not find an answer that existed.*

```
query: "ERR-4031 payment declined"
top result: general payment FAQ
expected page ERR-4031 troubleshooting: rank 10
```

Vector search treated the error code as a rare word and returned general text. Adding keyword search for exact matches and boosting title matches put the right page first, and a test set of real tickets guarded against future regressions.

## Common mistakes

- Using only vector search for codes and names
- Chunks too large or cut mid-sentence
- No metadata or access filters
- Never measuring retrieval quality

### Check yourself

**What does cosine similarity measure?**

Answer: The angle between vectors

**Why use an ANN index?**

Answer: Fast approximate search over many vectors

**Why add keyword search?**

Answer: Exact terms may be missed by vectors

**How judge retrieval?**

Answer: A test set of real queries

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 10. Retrieval-augmented generation (RAG)

13 minutes. Give a model fresh, private knowledge by retrieving relevant text and putting it in the prompt, with citations and checks against made-up answers.

A language model knows what was in its training data and nothing about your wiki, tickets or last week's decisions. Retrieval-augmented generation bridges that gap without retraining.

## What you will learn

- The RAG pipeline: ingest, retrieve, generate
- Prompting for grounded, cited answers
- Handling no results and conflicting sources
- Evaluating retrieval and answers separately

## The pipeline

1. **Ingest:** load documents, clean them, chunk them, embed them and store them with metadata.
2. **Retrieve:** embed the question, find the top chunks, and optionally rerank them.
3. **Generate:** build a prompt with the chunks and ask the model to answer from them.

```
def rag_answer(question, user, search, llm):
    chunks = search(question, top_k=6, filter={"groups": user.groups})
    if not chunks:
        return "I could not find this in the documentation."
    context = "\n\n".join(f"[{c.id}] {c.text}" for c in chunks)
    prompt = (
        "Use only the context to answer. Cite the ids you used like [id]. "
        "If the context does not answer the question, say so.\n\n"
        f"Context:\n{context}\n\nQuestion: {question}"
    )
    return llm(prompt)
```

## Grounded answers

Tell the model to use only the context, cite sources and admit gaps. Put the context in a clearly marked section. Handle conflicts by preferring newer or higher-authority documents, and show dates.

## Access control

Apply permissions at **retrieval time** with metadata filters, not only in the prompt. A model cannot keep a secret that you put in its context.

## Evaluate in two parts

Measure **retrieval** (did the right chunk appear in the top results) and **generation** (is the answer correct and faithful to the chunks). When quality drops, this tells you whether to improve chunking, search or prompts. Keep a regression set and update documents on a schedule, since stale sources create confident wrong answers.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: How Retrieval-augmented generation (RAG) fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **The pipeline** | Ingest: load documents, clean them, chunk them, embed them and store them with metadata. | Enforcing access only in the prompt. |
| **Grounded answers** | Tell the model to use only the context, cite sources and admit gaps. | Measuring only final answers, not retrieval. |
| **Access control** | Apply permissions at retrieval time with metadata filters, not only in the prompt. | A model cannot keep a secret that you put in its context. |
| **Evaluate in two parts** | Measure retrieval (did the right chunk appear in the top results) and generation (is the answer correct and faithful to the chunks). | No answer when nothing relevant is found. |

Flow: The pipeline to Grounded answers / Grounded answers to Access control / Access control to Evaluate in two parts

## Incident: The assistant that quoted a confidential file

**Incident:** *Retrieval ignored permissions and exposed a restricted document.*

```
user: contractor
retrieved: finance/salary-bands.md
answer included salary ranges
```

The index held every document and relied on the prompt to keep secrets. A contractor's question retrieved a restricted file. The team added group metadata and filtered at query time, tested access with sample users and logged which documents were used.

## Common mistakes

- Enforcing access only in the prompt
- No answer when nothing relevant is found
- Never refreshing stale documents
- Measuring only final answers, not retrieval

### Check yourself

**What does RAG add to a model?**

Answer: Retrieved private or current text

**Where enforce document access?**

Answer: At retrieval with filters

**What should the model do without relevant context?**

Answer: Say it cannot find the answer

**Why evaluate retrieval separately?**

Answer: It shows where quality is lost

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 11. Prompting, RAG or fine-tuning: choosing

12 minutes. Decide whether to change the prompt, add retrieval or fine-tune a model by looking at what is actually going wrong.

There are three main ways to improve a language model application, and people often reach for the most expensive one first. A short diagnosis of the failure tells you which to use.

## What you will learn

- What each approach changes
- Diagnosing the failure type
- Cost, risk and maintenance of each
- A sensible order of attempts

## What each one changes

- **Prompting:** instructions, examples, format and constraints. Fast, cheap and reversible.
- **RAG:** the knowledge the model sees. Good for private, large or changing facts, and for citations.
- **Fine-tuning:** the model's behavior. Good for a consistent style, strict format or a narrow task where examples are plentiful.

## Diagnose first

| Symptom | Likely fix |
|---|---|
| Ignores format or instructions | Better prompt, examples, structured output |
| Wrong or outdated facts about your data | RAG |
| Needs a house style at scale | Examples first, then fine-tuning |
| Too slow or costly for a simple task | Smaller model, possibly fine-tuned |
| Makes things up | Grounding, citations, allowing "I do not know" |

```
# a tiny harness: run the same cases against each variant
cases = load_jsonl("evals/cases.jsonl")
for name, system in [("prompt-v1", p1), ("prompt-v2", p2), ("rag", rag), ("tuned", tuned)]:
    score = sum(grade(system(c.input), c.expected) for c in cases) / len(cases)
    print(name, round(score, 3))
```

## Cost and maintenance

Fine-tuning needs good labeled data, a training and evaluation pipeline, and retraining when requirements or base models change. A tuned model can also lose general abilities. RAG needs a data pipeline and search quality work, but updates are immediate when documents change.

## Order of attempts

1. Write an evaluation set from real examples.
2. Improve the prompt and examples.
3. Add retrieval if facts are the problem.
4. Consider fine-tuning only for a measured gap, with enough clean data.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs marga.eknathalabs.com

### Diagram: How Prompting, RAG or fine-tuning: choosing fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **What each one changes** | Prompting: instructions, examples, format and constraints. | Fine-tuning to teach changing facts. |
| **Diagnose first** | . | Fine-tuning with a small, messy dataset. |
| **Cost and maintenance** | Fine-tuning needs good labeled data, a training and evaluation pipeline, and retraining when requirements or base models change. | No evaluation set before changing anything. |
| **Order of attempts** | Write an evaluation set from real examples. | Skipping prompt improvements. |

Flow: What each one changes to Diagnose first / Diagnose first to Cost and maintenance / Cost and maintenance to Order of attempts

## Incident: Fine-tuned on yesterday's prices

**Incident:** *A tuned model gave outdated answers because knowledge was baked into the weights.*

```
question: price of plan Pro?
answer: $29/month
current price (source of truth): $39/month
```

The team had put changing facts into training data. Every price change made the model wrong until the next training run. They moved facts to retrieval from the pricing service and kept fine-tuning only for tone and output format.

## Common mistakes

- Fine-tuning to teach changing facts
- No evaluation set before changing anything
- Skipping prompt improvements
- Fine-tuning with a small, messy dataset

### Check yourself

**Which approach suits changing facts?**

Answer: RAG

**Which is cheapest and most reversible?**

Answer: Prompting

**What suits a consistent style at scale?**

Answer: Examples, then fine-tuning

**What first?**

Answer: An evaluation set

MARGA by EknathaLabs
marga.eknathalabs.com

# Lesson 12. LLM safety, privacy and data handling

13 minutes. Handle data safely in language model applications: redact personal data, control retention, defend against prompt injection and log responsibly.

Language model applications mix untrusted text, private data and powerful tools. Most incidents come from a few predictable mistakes: sending too much, logging too much and trusting text that came from outside.

## What you will learn

- Classifying and redacting data before it reaches a model
- Provider settings: retention, training and region
- Prompt injection and untrusted content
- Safe logging and output handling

## Data in prompts

Decide which classes of data may go to which provider. Redact secrets, tokens and personal identifiers before sending, replacing them with placeholders and restoring them afterwards if needed.

```
import re

PATTERNS = {
    "email": re.compile(r"[\w.+-]+@[\w-]+\.[\w.]+"),
    "token": re.compile(r"\b(?:sk|ghp|AKIA)[A-Za-z0-9_-]{16,}\b"),
}

def redact(text):
    for name, rx in PATTERNS.items():
        text = rx.sub(f"[{name.upper()}]", text)
    return text
```

## Provider and retention

Check contract terms: whether inputs are used for training, how long they are retained, where they are processed and what audit options exist. Use zero or short retention for sensitive workloads and keep a record of approved providers.

## Prompt injection

Text from emails, web pages and documents can contain instructions that try to hijack the model. Treat all such content as **data, not commands**: put it in a delimited section, tell the model not to follow instructions inside it, and, more importantly, limit what the application can do. If a model reads untrusted text, do not give it the ability to send email, move money or read secrets without a human approval step.

## Logging and outputs

Logs help debugging and can leak data. Redact before logging, restrict access and set retention. Validate model output before using it: escape it before displaying, check JSON against a schema and never execute generated code or SQL without review and sandboxing.

## How the parts fit together

The lesson in one picture: each box is a section of this lesson, in the order you meet it. Select a box to see what it does and the mistake that usually goes with it.

MARGA by EknathaLabs
marga.eknathalabs.com

### Diagram: How LLM safety, privacy and data handling fits together

| Part | What it does | What goes wrong |
|---|---|---|
| **Data in prompts** | Decide which classes of data may go to which provider. | Logging full prompts forever. |
| **Provider and retention** | Check contract terms: whether inputs are used for training, how long they are retained, where they are processed and what audit options exist. | Skipping provider retention checks. |
| **Prompt injection** | Text from emails, web pages and documents can contain instructions that try to hijack the model. | Giving a model that reads untrusted text powerful tools. |
| **Logging and outputs** | Logs help debugging and can leak data. Redact before logging, restrict access and set retention. | Using output without validation. |

Flow: Data in prompts to Provider and retention / Provider and retention to Prompt injection / Prompt injection to Logging and outputs

## Incident: Customer emails in the debug logs

**Incident:** *Prompt logging stored private data with weak access controls.*

```
log: prompt="Hi, my card 4111 1111 1111 1111 declined, call 555-0143"
retention: unlimited
readers: all engineering (412 people)
```

The debug logging was enabled to improve prompts and was never reviewed. Logs held card numbers and phone numbers. The team redacted before logging, limited access to a small group, set a 14-day retention and added data checks to code review.

## Common mistakes

- Logging full prompts forever
- Giving a model that reads untrusted text powerful tools
- Skipping provider retention checks
- Using output without validation

### Check yourself

**What is prompt injection?**

Answer: Untrusted text trying to give the model instructions

**What reduces injection impact most?**

Answer: Limiting what the app can do

**What should happen before logging prompts?**

Answer: Redact sensitive data

**How treat generated SQL or code?**

Answer: As untrusted until reviewed and sandboxed

MARGA by EknathaLabs
marga.eknathalabs.com

# Cheat sheet: LLM fundamentals

Tokens, context limits, API calls, sampling, cost, caching, evaluation and a debugging checklist for LLM features.

## A first API call

```
import os
import anthropic

MODEL = os.environ["MODEL_ID"]
client = anthropic.Anthropic()          # reads ANTHROPIC_API_KEY
resp = client.messages.create(
    model=MODEL,
    max_tokens=500,
    system="You are a concise assistant.",
    messages=[{"role": "user", "content": "Explain a 502 error."}],
)
print(resp.content[0].text, resp.stop_reason, resp.usage)
```

> **Warning:** *Never put API keys in code, front-end JavaScript, logs or screenshots. Use a secret manager and rotate on exposure.*

## Tokens and limits

```
if resp.stop_reason == "max_tokens":
    raise RuntimeError("reply was cut off")
print(resp.usage.input_tokens, resp.usage.output_tokens)
```

One token is about four English characters. Input, history, documents, tool results and the answer all count toward the context window.

> **Warning:** *A reply cut off by the length limit is incomplete. Truncated JSON will not parse, so always check the stop reason.*

## Sampling

```
client.messages.create(model=MODEL, max_tokens=300, temperature=0.0,
                       messages=[{"role": "user", "content": "Classify: " + text}])
```

Low temperature for extraction, classification and code. Higher for brainstorming.

> **Warning:** *Temperature 0 does not guarantee identical output. Test properties and pass rates, not exact sentences.*

## Errors and retries

```
import random, time

def with_retry(call, tries=4):
    for n in range(tries):
        try:
            return call()
        except anthropic.RateLimitError:
            time.sleep(2 ** n + random.random())
    raise RuntimeError("rate limited")
```

Retry 429 and 5xx with backoff and jitter. Do not retry 400, 401 or 403.

## Cost estimate

```
def cost(inp, out, price_in, price_out):
    return inp / 1e6 * price_in + out / 1e6 * price_out
```

Output tokens usually cost more than input tokens. Use the current price list.

> **Warning:** *Agents and retry loops can spend a month of budget overnight. Set provider spend limits, alerts and a step cap.*

MARGA by EknathaLabs marga.eknathalabs.com

## Caching and streaming

```
with client.messages.stream(model=MODEL, max_tokens=500, messages=msgs) as s:
    for text in s.text_stream:
        print(text, end="", flush=True)
```

Put stable content first and changing content last to benefit from prompt caching.

## Evaluate

```
{"id": "t-014", "input": "Card charged twice", "expect": {"category": "billing"}}
```

Keep 30 to 100 real cases, run every candidate several times, and compare pass rate, cost and latency. Re-run on any change.

## Debug a bad answer

```
jq '{id, model, stop_reason, usage, error}' req.json
jq -r '.prompt' req.json | head -60
```

**Warning:** *Model output is untrusted. Validate it before it reaches a shell, a database or a customer.*

MARGA by EknathaLabs
marga.eknathalabs.com